

DESIGN PATTERN AND ANALYSIS-SINGLETON PATTERN

Exercise 1: Implementing the Singleton Pattern

Scenario:

You need to ensure that a logging utility class in your application has only one instance throughout the application lifecycle to ensure consistent logging.

Steps:

Create a New Java Project:

Create a new Java project named **SingletonPatternExample**.

Define a Singleton Class:

Create a class named **Logger** that has a private static instance of itself.

Ensure the constructor of **Logger** is private.

Provide a public static method to get the instance of the **Logger** class.

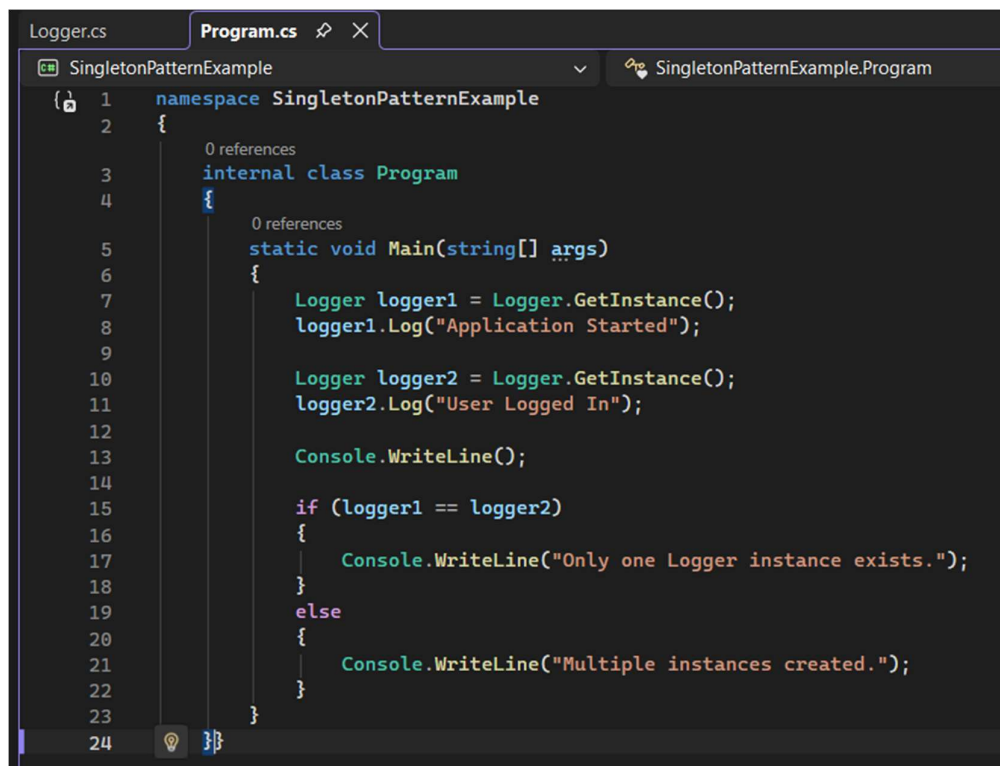
Implement the Singleton Pattern:

Write code to ensure that the **Logger** class follows the Singleton design pattern.

Test the Singleton Implementation:

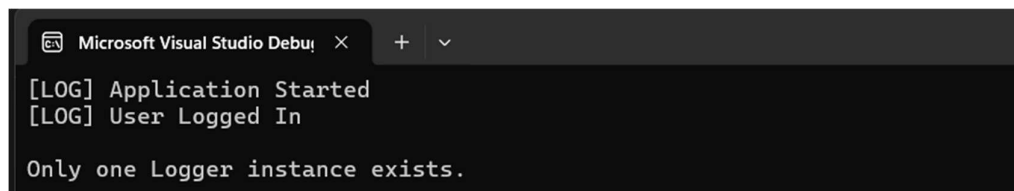
Create a test class to verify that only one instance of **Logger** is created and used across the application.

```
1 namespace SingletonPatternExample
2 {
3     8 references
4     public sealed class Logger
5     {
6         private static Logger? instance;
7
8         1 reference
9         private Logger()
10        {
11        }
12
13        2 references
14        public static Logger GetInstance()
15        {
16            if (instance == null)
17            {
18                instance = new Logger();
19            }
20
21            return instance;
22        }
23
24        2 references
25        public void Log(string message)
26        {
27            Console.WriteLine($"[LOG] {message}");
28        }
29    }
30 }
```



```
1 namespace SingletonPatternExample
2 {
3     0 references
4     internal class Program
5     {
6         0 references
7         static void Main(string[] args)
8         {
9             Logger logger1 = Logger.GetInstance();
10            logger1.Log("Application Started");
11
12            Logger logger2 = Logger.GetInstance();
13            logger2.Log("User Logged In");
14
15            Console.WriteLine();
16
17            if (logger1 == logger2)
18            {
19                Console.WriteLine("Only one Logger instance exists.");
20            }
21            else
22            {
23                Console.WriteLine("Multiple instances created.");
24            }
25        }
26    }
27 }
```

OUTPUT



```
Microsoft Visual Studio Debug Console
[LOG] Application Started
[LOG] User Logged In

Only one Logger instance exists.
```